

DAY 3

Module 7

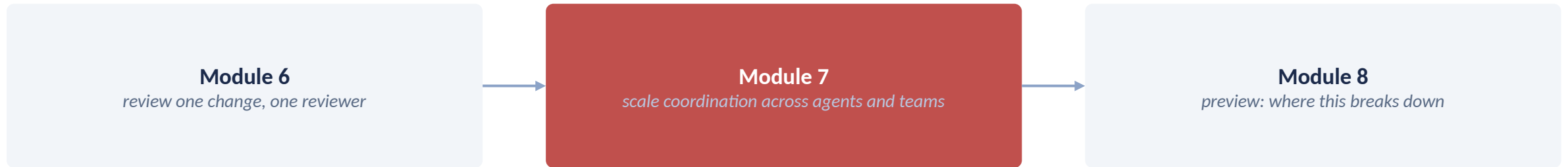
Multi-Agent Workflows and Team Adoption

Coordinating parallel work, and keeping an audit trail across teams and toolchains.

What You'll Learn

- 1 Coordinate multiple agents or subagents across a task graph
- 2 Decide when to parallelize versus sequence work
- 3 Apply repository conventions and governance across teams and toolchains

Everything So Far Assumed One Agent, One Reviewer



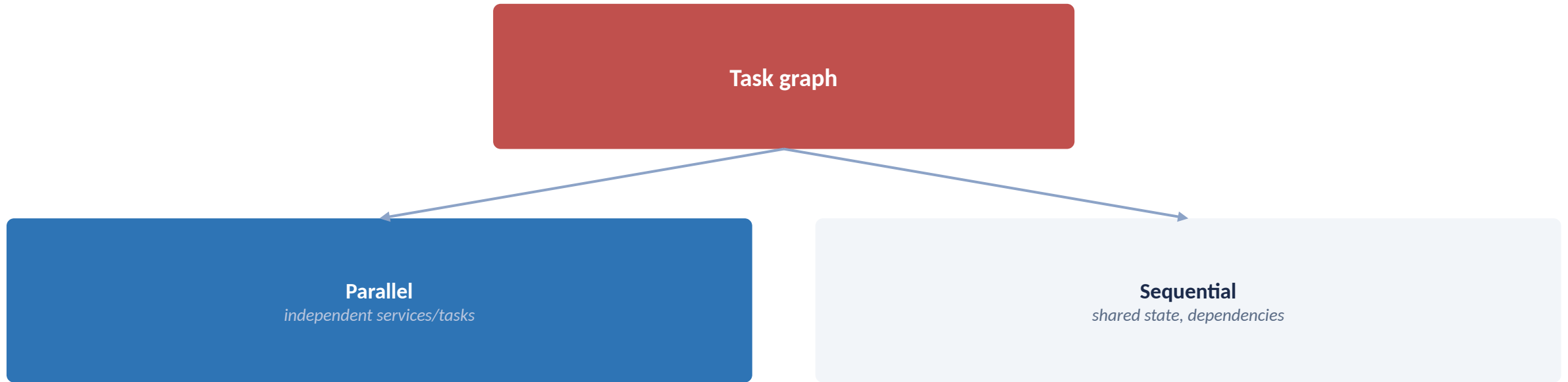
Every gate and review habit from Modules 5 and 6 still applies once there are multiple agents and multiple people. What changes is coordination: who's touching what, and whether the audit trail holds up across toolchains.

PART 1 • COORDINATION

Coordinating Parallel Work

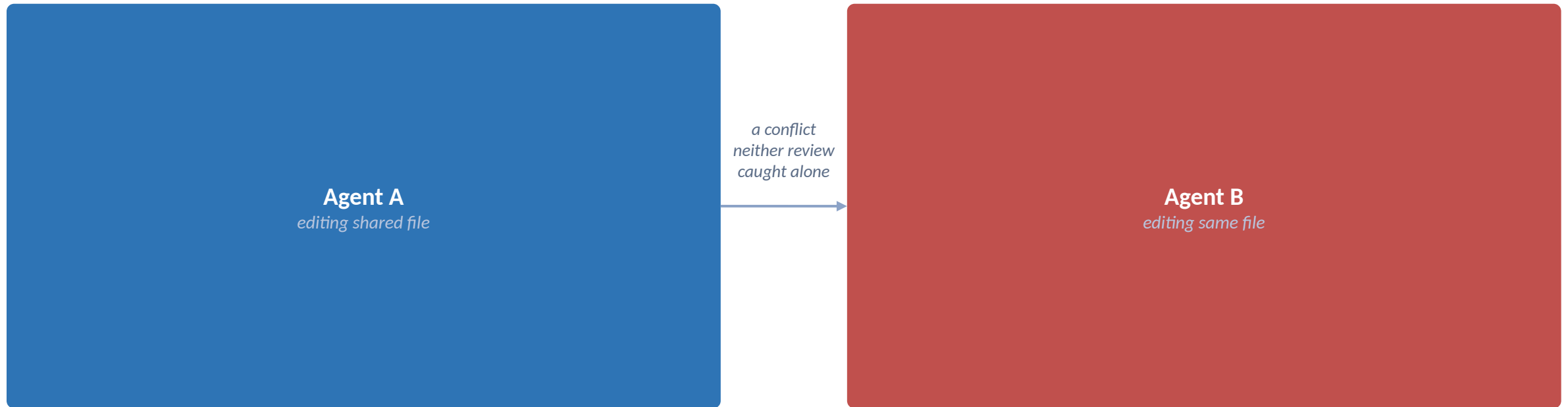
Parallelism is a property of the work, not a default setting

Parallelize or Sequence?



The decision is made per task, not assumed by default.

Uncoordinated Parallel Agents Create Their Own Kind of Chaos



Visibility has to be designed in before you parallelize.

Checkpoint: What decides whether two tasks can run in parallel?

- Whether they share state or files, not whether they happen to be unrelated in the spec.
- The task graph from planning is what makes this decision answerable, not a guess made mid-execution.
- Uncoordinated parallelism creates conflicts that single-change review, from Module 6, won't catch alone.

PART 2 • APPLYING THE CONCEPTS

Explicit Orchestration

How each framework actually coordinates multiple streams of work

Workflows: Explicit Orchestration

specify workflow run

YAML-defined workflows chain commands, prompts, and shell steps into repeatable sequences, with gate steps for human checkpoints, so a multi-step process runs the same way every time instead of being reassembled manually.

Parallel primitive

Fan-out and fan-in step types dispatch a step across a list of items and aggregate the results, a direct primitive for coordinating parallel agent work across a task graph, so parallel work doesn't have to be hand-rolled each time.

Audit trail

Every run persists state (state.json, log.jsonl) and can pause and resume from the exact point of interruption, doubling as an audit trail, so a long-running workflow doesn't have to restart from scratch after an interruption.

Bulk-Archive, Not Orchestration

```
/opsx:bulk-archive
```

Coordinates multiple parallel change streams at once, detecting and resolving spec conflicts between them by checking what's actually implemented, which is the closest OpenSpec gets to explicit multi-agent orchestration.

Worth knowing

OpenSpec's beta "Stores" feature extends planning across multiple repos or teams, for organizations working past a single-repo scope, worth knowing about even though it's still evolving.

Important distinction

Genuine agent-to-agent delegation is a capability of the underlying coding agent (Claude Code subagents, Copilot Fleet mode), not a feature either SDD framework owns directly, so this is a distinction worth keeping straight when evaluating either tool.

Module 7 at a Glance

	GitHub Spec Kit	OpenSpec
Orchestration primitives	fan-out / fan-in, gates, resumable state	None (conflict-aware bulk archiving only)
Multi-stream support	Workflow YAML with conditional logic	/opsx:bulk-archive
Agent delegation	Not framework-owned (agent capability)	Not framework-owned (agent capability)

Checkpoint: What does neither framework manage for you?

- Genuine agent-to-agent delegation, that's a capability of the underlying coding agent, not the SDD framework.
- Spec Kit gives explicit orchestration primitives (fan-out/fan-in, gates); OpenSpec's support is lighter-weight, conflict detection after the fact.
- Both frameworks still expect a human to decide what actually gets parallelized.

PART 3 • AT TEAM SCALE

Conventions and Governance

Governance has to hold up across teams using different toolchains

Conventions and Governance



Governance has to hold up across teams using different toolchains.

Making Governance Hold Across Teams

One constitution, many services

A shared constitution (Module 3) sets organization-wide non-negotiables; each service or team can extend it with local conventions, but shouldn't contradict it. Name the split explicitly, don't leave it implicit.

Different toolchains, same audit standard

One team on Spec Kit and another on OpenSpec is fine, as long as both can answer 'what changed, why, and who approved it' the same way. The tool differs; the audit standard shouldn't.

Issues and PRs as the shared record

Regardless of which SDD framework generated a change, the issue tracker and PR history remain the cross-team source of truth that any reviewer, human or agent, can reconstruct from later.

Checkpoint: What has to stay constant across teams on different toolchains?

- The audit standard: what changed, why, and who approved it, answerable the same way regardless of tool.
- A shared constitution's non-negotiables, even as local conventions vary by service or team.
- Issues and PRs as the cross-team record, independent of which SDD framework produced the change.

Common Pitfalls

Assuming independence instead of checking for it

Marking tasks parallel because they look unrelated in the spec, without verifying they don't share state or files, is how the Agent A/Agent B conflict happens.

Letting tool choice fragment the audit trail

If different teams' toolchains produce records in incompatible formats, no one can reconstruct a cross-team decision later, exactly when it matters most.

Treating orchestration primitives as optional polish

Hand-rolling fan-out/fan-in coordination instead of using the framework's built-in primitives reintroduces the exact chaos those primitives exist to prevent.

Key Takeaways

- Parallelism is a property of the work, not a default setting, so it has to be decided case by case, not assumed.
- Spec Kit's workflow engine gives you explicit orchestration primitives; OpenSpec's support is lighter-weight and less structured.
- Delegation to a second agent is your coding agent's job, not the SDD framework's, so don't expect either framework to manage that hand-off for you.
- Governance holds across teams when the audit standard stays fixed even as the toolchain varies. Module 8 closes the course by naming where these habits tend to break down.